

ML Prediction of CO2 Adsorption on Biomass-Derived MgO-Modified Porous Carbons

Context

Goal: Produce a publishable ML paper predicting CO2 uptake (mg/g) on biomass-derived MgO-modified porous carbons using literature data.

Dataset: 0514biochar-MgOdata.xlsx — 341 samples, 17 columns (two-row header), literature compilation. Target: CO2 uptake (1.32–292.67 mg/g).

Key data issues: High missingness in several columns; categorical features with many levels; text columns containing numerical parameters that need regex parsing; significant class imbalance in precursors.

Project Structure

```
NEW-ML-MgOBC/
├── data/
│   ├── raw/0514biochar-MgOdata.xlsx    # Never modified
│   └── processed/                       # Generated CSVs
├── src/
│   ├── config.py                       # Paths, constants, seeds
│   ├── data_loader.py                  # Load & merge 2-row header
│   └── preprocessing.py                 # Missing value imputation, en
coding
├── feature_engineering.py               # Regex parsing, composite fea
tures
├── models.py                           # Model factory functions (6 m
odels)
├── train.py                            # Nested CV + Optuna HP tuning
├── evaluate.py                          # Metrics, model comparison
├── shap_analysis.py                     # SHAP computation & visualiza
tion
├── plotting.py                          # Publication figures (300 DPI)
├── notebooks/                           # 7 notebooks (EDA → SHAP)
├── outputs/figures/ + tables/           # Final paper assets
└── requirements.txt
```

Step 1: Data Loading & Cleaning

File: src/data_loader.py

- Read Excel with header=None, skip first 2 rows (merged header)

- Assign clean column names: carbon_precursors, MgO_precursors, Mg_loading_method, Activation1, Activation2, Carbonization1, Carbonization2, Post_treatment, SBET_m2_g, Vtotal_cm3_g, Vmicro_cm3_g, MgO_crystallite_size_nm, MgO_mass_ratio, temperature_C, pressure_bar, CO2_uptake_mg_g, References
- **Drop:** References (literature citation, not a feature)
- Convert all numeric columns via `pd.to_numeric(errors='coerce')`
- Normalize Unicode variants in MgO_precursors (Mg(CH3COO)2·4H2O vs Mg(CH3COO)2·4H2O)

NaN 语义修正 — 以下列的 NaN 不是数据缺失，而是“未执行该步骤”： -

Activation1, Activation2, Carbonization1, Carbonization2, Post_treatment 的 NaN 全部替换为 "none" - 这意味着 **Activation2** 和 **Post_treatment** 保留使用，不丢弃

Column renaming for consistency: - Mg_loading_method: merge “impregnation” and “wetness impregnation” (essentially the same method)

Step 2: Missing Value Handling

File: src/preprocessing.py

真正缺失的列（仅数值列存在数据缺失）：

Column	Missing %	Strategy
SBET_m2_g	4.4%	KNNImputer (k=5) — 低缺失率
Vtotal_cm3_g	4.4%	KNNImputer (k=5)
Vmicro_cm3_g	42.2%	IterativeImputer (BayesianRidge, max_iter=20) — 对 CO2 吸附机理重要
MgO_crystallite_size_nm	72.1%	双轨策略: (A) 删除该特征; (B) IterativeImputer。比较后以 A 为主

非缺失列（NaN 已在上一步转为 “none” 类别）： - Activation1, Activation2, Carbonization1, Carbonization2, Post_treatment - NaN=“none” 代表该步骤未执行，是有意义的工艺信息

Critical: Fit all imputers on training set only, transform test set to avoid data leakage.

Robustness check: Train secondary model on 197 Vmicro-complete samples only (no imputation) and compare R^2 .

Step 3: Feature Engineering

File: src/feature_engineering.py

3a. Parse numerical values from text columns

核心思路：5个工艺列中，非“none”的文本包含温度、时间等数值信息，用正则提取为数值特征；同时保留分类特征表示工艺类型。

From Activation1 (e.g., “KOH activation at 700°C for 2 h”, “none”): - 提取 → act1_temp_C, act1_duration_h - 分类 → act1_type: none / KOH / steam / hydrothermal

From Activation2 (332 “none”, 9 “H3PO4 activation at 700°C for 1 h”): - 提取 → act2_temp_C, act2_duration_h - 分类 → act2_type: none / H3PO4

From Carbonization1 (e.g., “microwave heating at 160 °C for 1 h”, “fast pyrolysis at 600°C”, “none”): - 提取 → carb1_temp_C, carb1_duration_h - 分类 → carb1_type: none / conventional / microwave / fast_pyrolysis

From Carbonization2 (167 “none”, 157 “600 °C for 2 h”, 17 “500 °C for 15 min”): - 提取 → carb2_temp_C, carb2_duration_h - 分类 → carb2_type: none / 500C_15min / 600C_2h

From Post_treatment (326 “none”, 6 “without rinsing”, 5 “with rinsing”, 4 “sonication at 60% amplitude for 1 min”): - 分类 → post_treatment_type: none / without_rinsing / with_rinsing / sonication

3b. Domain-informed composite features

Feature	Formula	Rationale
Vmeso_cm3_g	Vtotal - Vmicro	Mesopore volume
microporosity	Vmicro / Vtotal	Key for CO2 physisorption
MgO_surface_density	MgO_mass_ratio / (SBET / 1000)	MgO per unit surface area
T_lnP	temperature * ln(pressure + 0.01)	Combined adsorption condition term

3c. Final feature set (~25 features)

Numerical (16-18): SBET, Vtotal, Vmicro, MgO_mass_ratio, temperature, pressure, act1_temp, act1_duration, act2_temp, act2_duration, carb1_temp, carb1_duration, carb2_temp, carb2_duration, Vmeso, microporosity, MgO_surface_density, T_lnP — optionally MgO_crystallite_size

Categorical (8) (OrdinalEncode for trees, TargetEncode for SVR/GPR): carbon_precursors, MgO_precursors, Mg_loading_method, act1_type, act2_type, carb1_type, carb2_type, post_treatment_type

Step 4: ML Models (6 total)

File: src/models.py

1. **Random Forest** — baseline, robust to outliers
2. **XGBoost** — expected best performer, native NaN handling
3. **LightGBM** — fast, often competitive with XGBoost
4. **GradientBoostingRegressor** (sklearn) — traditional GBDT
5. **SVR** (RBF kernel) — requires StandardScaler + log1p target transform
6. **Gaussian Process Regression** — provides prediction uncertainty

Two preprocessing pipelines: - **Pipeline A (tree models):** OrdinalEncoder for categoricals, no scaling needed - **Pipeline B (SVR/GPR):** TargetEncoder for high-cardinality categoricals, StandardScaler for all features, log1p(target)

Step 5: Model Evaluation

File: src/train.py, src/evaluate.py

- **Nested CV:** Outer 5-fold, inner 3-fold for unbiased performance estimation
- **Group-aware:** Use GroupKFold with study_group (clustered from [carbon_precursor, MgO_precursor, method] combo) to detect batch effects
- **Metrics:** R^2 , RMSE (mg/g), MAE (mg/g), MAPE (%)
- **HP tuning:** Optuna TPE, 100 trials per model, minimize RMSE
- **Final model:** Train best model on full training data after HP tuning
- **Ablation study:** Compare 4 feature sets (with/without MgO_crystallite_size, parsed vs raw text, Vmicro-imputed vs Vmicro-complete-only)

Step 6: SHAP Interpretability

File: src/shap_analysis.py

- **TreeExplainer** for XGBoost/LightGBM (exact SHAP)
- **KernelExplainer** for SVR/GPR (on 100-sample subset)
- **Figures:** beeswarm summary, bar plot, dependence plots for top 5 features, waterfall for best/worst predictions

Step 7: Paper Figures

File: src/plotting.py (all 300 DPI, consistent seaborn style)

Fig Content

-
- | | |
|---|---|
| 1 | Feature correlation heatmap (clustermap) |
| 2 | CO2 uptake distribution by precursor type |
| 3 | Model performance comparison bar chart (R^2 + RMSE) |
| 4 | Parity plot: predicted vs experimental, colored by SBET |
| 5 | SHAP summary beeswarm |
| 6 | SHAP dependence plots for top 3 features |
| 7 | Residual analysis (2×2: residuals vs fitted, Q-Q, histogram, Cook's distance) |

8 Partial dependence + ICE for top 4 features

Step 8: Paper Outline

Title: “Machine Learning Prediction of CO₂ Adsorption Capacity on Biomass-Derived MgO-Modified Porous Carbons with SHAP Interpretability”

Sections: Introduction → Methods (Dataset, Preprocessing, Feature Engineering, ML Algorithms, Evaluation) → Results & Discussion (EDA, Model Comparison, SHAP Interpretation, Ablation Study, Limitations) → Conclusion

Implementation Order

1. `config.py` + `data_loader.py`
 2. `feature_engineering.py` + `preprocessing.py`
 3. `models.py` + `train.py` + `evaluate.py`
 4. `shap_analysis.py` + `plotting.py`
 5. Notebooks (sequential: 01→07)
 6. Generate all figures/tables for paper
-

Verification

- Run nested CV and confirm XGBoost/LightGBM $R^2 > 0.80$ on test set
- Verify SHAP top features are consistent across XGBoost, LightGBM, and RF
- Check GroupKFold vs regular KFold R^2 gap; if >0.05 , batch effects present → discuss
- Compare Vmicro-imputed vs Vmicro-complete-only models; R^2 difference should be <0.10
- All figures generated at 300 DPI in `outputs/figures/`